

SIMATS ENGINEERING

ASSESSMENT TOOL 3

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 40%

QUESTIONS: 3

Duration: 1 Hour
Total Marks:30

Annexure A

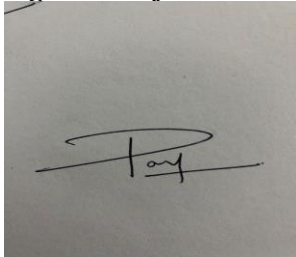
Dry Run Evaluation

Code of Conduct:

I PAVAN.P / 192411017 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

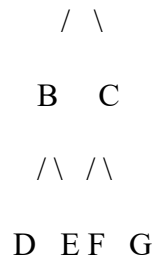


Annexure A

Dry Run Evaluation

1. Question 1 – Breadth-First Search (BFS)

Consider the following graph: A



Task: Starting from node **A**, perform a **Breadth-First Search (BFS)**. Complete the following table.

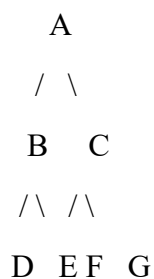
Iteration Queue Visited Node

1
2
3
4
5
6
7

Questions:

1. Write the BFS traversal order.
2. Show the queue contents after each iteration.

2. Depth-First Search (DFS) Using the same graph,



Iteration	Stack	Visited Node
1		
2		
3		
4		
5		
6		
7		

Perform **Depth-First Search (DFS)** starting from **A**.

Complete the table.

3. Initial State :

1 3 6
5 0 2
4 7 8

Goal State :

1 2 3
4 5 6

7 8 0

Task

Trace **Breadth-First Search (BFS)** until the goal state is reached.

Fill in the table.

Iteration Current State Queue Before Expansion Queue After Expansion

1

2

3

4

5

Questions

1. Which node is expanded in each iteration?
2. How many states are generated in total?
3. Write the complete solution path from the initial state to the goal state.
4. Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
Understanding of Search Problem	20	Clearly understands the search problem, initial state, goal	Understands the problem with minor	Demonstrates partial understanding of	Shows little or no understanding

Criteria	Weightage (%)	Level 4 – Exemplary (4)	Level 3 – Proficient (3)	Level 2 – Developing (2)	Level 1 – Beginning (1)
		state, and search strategy.	misunderstandings.	the search problem.	of the search problem.
State Expansion and Dry Run Execution	30	Correctly traces every iteration, expands nodes accurately, and maintains the Queue/Stack/Priority Queue without errors.	Performs most iterations correctly with minor mistakes in state expansion or data structure updates.	Performs only some iterations correctly; several errors in tracing or node expansion.	Unable to correctly perform the dry run or expand states.
Application of Search Algorithm	20	Applies the selected algorithm (BFS/DFS/UCS) correctly, following all algorithmic rules.	Applies the algorithm correctly with a few procedural errors.	Applies only basic algorithm steps with noticeable mistakes.	Fails to apply the correct search algorithm.
Accuracy of Solution Path	20	Identifies the correct traversal order/solution path and goal state with proper justification.	Solution path is mostly correct with minor errors.	Partially correct solution path; goal may not be reached correctly.	Incorrect or incomplete solution path.
Presentation and Logical Reasoning	10	Queue/Stack/Priority Queue updates, state diagrams, and explanations are neat, organized, and logically presented.	Presentation is clear with minor organizational issues.	Limited explanation and organization; reasoning is partially clear.	Poor presentation with unclear or missing reasoning.

1. ANSWER

PYTHON CODING: -

```
from collections import deque
```

```
# Graph representation
```

```
graph = {
```

```

'A': ['B', 'C'],
'B': ['D', 'E'],
'C': ['F', 'G'],
'D': [],
'E': [],
'F': [],
'G': []
}

visited = set()
queue = deque(['A'])

print("Iteration\tQueue\t\tVisited Node")
print("-" * 50)

iteration = 1
bfs_order = []

while queue:
    node = queue.popleft()

    if node not in visited:
        visited.add(node)
        bfs_order.append(node)

        # Add adjacent nodes
        for neighbor in graph[node]:
            if neighbor not in visited:
                queue.append(neighbor)

        # Display queue after visiting the node
        if len(queue) == 0:
            queue_display = "Empty"
        else:
            queue_display = list(queue)

        print(f"{iteration}\t\t{queue_display}\t\t{node}")
        iteration += 1

print("\nBFS Traversal Order:")
print("-> ".join(bfs_order))

```

OUTPUT: -

Iteration	Queue	Visited Node
1	['B', 'C']	A
2	['C', 'D', 'E']	B
3	['D', 'E', 'F', 'G']	C
4	['E', 'F', 'G']	D
5	['F', 'G']	E
6	['G']	F
7	Empty	G

BFS Traversal Order:
A -> B -> C -> D -> E -> F -> G

2. ANSWER

PYTHON CODING :-

DFS Program using Recursion

```
graph = { 'A': ['B', 'C'], 'B': ['D', 'E'], 'C': ['F', 'G'], 'D': [], 'E': [], 'F': [], 'G': [] }
```

```
visited = [] iteration = 1
```

```
print("Iteration\tVisited Node\tVisited List") print("-" * 50)
```

```
def dfs(node): global iteration
```

```
    visited.append(node)
```

```
    print(f"{iteration}\t\t\t{node}\t\t\t{visited}")
```

```
    iteration += 1
```

```
    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(neighbor)
```

```
dfs('A')
```

```
print("\nDFS Traversal Order:") print(" -> ".join(visited))
```

OUTPUT: -

Iteration	Visited Node	Visited List
1	A	['A']
2	B	['A', 'B']
3	D	['A', 'B', 'D']
4	E	['A', 'B', 'D', 'E']
5	C	['A', 'B', 'D', 'E', 'C']
6	F	['A', 'B', 'D', 'E', 'C', 'F']
7	G	['A', 'B', 'D', 'E', 'C', 'F', 'G']

DFS Traversal Order:

A -> B -> D -> E -> C -> F -> G

3. ANSWER

PYTHON CODING :

```
from collections import deque
```

```
# Initial State
```

```
initial = (
    (1, 3, 6),
    (5, 0, 2),
    (4, 7, 8)
)
```

```
# Goal State
```

```
goal = (
    (1, 2, 3),
```

```

    (4, 5, 6),
    (7, 8, 0)
)

# Possible moves
moves = [(-1,0), (1,0), (0,-1), (0,1)]

# Find position of blank (0)
def find_blank(state):
    for i in range(3):
        for j in range(3):
            if state[i][j] == 0:
                return i, j

# Generate next states
def get_neighbors(state):
    x, y = find_blank(state)
    neighbors = []

    for dx, dy in moves:
        nx, ny = x + dx, y + dy

        if 0 <= nx < 3 and 0 <= ny < 3:
            temp = [list(row) for row in state]
            temp[x][y], temp[nx][ny] = temp[nx][ny], temp[x][y]
            neighbors.append(tuple(tuple(row) for row in temp))

    return neighbors

# BFS Algorithm
def bfs(initial, goal):
    queue = deque([(initial, [])])
    visited = set()

    while queue:
        state, path = queue.popleft()

        if state in visited:
            continue

        visited.add(state)

        if state == goal:
            return path + [state], len(visited)

        for neighbor in get_neighbors(state):
            if neighbor not in visited:
                queue.append((neighbor, path + [state]))

    return None, len(visited)

```



```
# Print puzzle
def print_state(state):
    for row in state:
        print(*row)
    print()

# Run BFS
solution, total = bfs(initial, goal)

if solution:
    print("Goal Found!\n")
    print("Solution Path:\n")

    for step, state in enumerate(solution):
        print("Step", step)
        print_state(state)

    print("Total Moves =", len(solution)-1)
    print("Total States Expanded =", total)

else:
    print("No Solution Found.")
```

OUTPUT: -

Goal Found!

Solution Path:

Step 0

1 3 6

5 0 2

4 7 8

Step 1

1 3 6

5 2 0

4 7 8

Step 2

1 3 0

5 2 6

4 7 8

Step 3

1 0 3

5 2 6

4 7 8

Step 4

1 2 3

5 0 6

4 7 8

Step 5

1 2 3

0 5 6

4 7 8